PYSPARK COMPLETE NOTES

PHASE 7

Spark SQL & Pipelines

Query DataFrames with SQL, then see how real pipelines are built: medallion layers, jobs and safe reruns.

YOU WILL

- ✓ Register a DataFrame as a temporary view
- ✓ Run SQL in a %sql cell and with spark.sql()
- ✓ Translate between the DataFrame API and SQL
- ✓ Compare a manual notebook run with a scheduled Databricks Job
- ✓ Describe Bronze, Silver and Gold, batch vs streaming and full vs incremental loads
- ✓ Make a pipeline idempotent and handle failed runs

START HERE IF

Phases 2-6 (so you recognise the operations in SQL form and the layers they belong to).

Built from a Databricks PySpark tutorial notebook and its course outline.
Clickable contents, links and bookmarks work in any PDF viewer.

Phase 7 · Spark SQL & Pipelines

Query DataFrames with SQL, then see how real pipelines are built: medallion layers, jobs and safe reruns.

YOU WILL BE ABLE TO

- ✓ Register a DataFrame as a temporary view
- ✓ Run SQL in a %sql cell and with spark.sql()
- ✓ Translate between the DataFrame API and SQL
- ✓ Compare a manual notebook run with a scheduled Databricks Job
- ✓ Describe Bronze, Silver and Gold, batch vs streaming and full vs incremental loads
- ✓ Make a pipeline idempotent and handle failed runs

START HERE IF

Phases 2-6 (so you recognise the operations in SQL form and the layers they belong to).

IN THIS PDF

Explanations, diagrams and code from the notebook · sections tagged **Beyond the notebook** · a decoder for every term · interview questions · a data-engineering lens · a self-test with answers · a glossary

How to read the boxes

IN PLAIN WORDS
A jargon-free explanation of the idea.

DECODE THE CODE
Meaning of every function, option and symbol in the code above (first time it appears).

TIP
A habit that makes your code cleaner, faster or safer.

WATCH OUT
A trap that gives a wrong or surprising result.

NOTEBOOK OBSERVATION
Something specific noticed in the course notebook.

DATA ENGINEERING LENS
How the skill is used in real pipelines (summary page at the end).

EXTRA
Extra knowledge that is not in the notebook.

BEYOND THE NOTEBOOK
A tag under a heading: new material added to the notebook's topics, ending with an Interview corner.

CONTENTS (click an entry to jump to it)

7.1 createTempView() . . . 3
7.2 Running SQL: the %sql cell . . . 4
7.3 spark.sql(): SQL that returns a DataFrame . . . 4
7.4 Running a notebook vs running in production . . . 5
7.5 Databricks Jobs and orchestration . . . 6
7.6 Medallion architecture: Bronze, Silver and Gold . . . 8
7.7 Batch vs streaming . . . 10
7.8 Full vs incremental loads . . . 11
7.9 Idempotency and safe reruns . . . 12
7.10 Lineage, error handling and failed runs . . . 13
7.11 What the course notebook does and what is still ahead . . . 15
From notebook to pipeline . . . 17
Glossary . . . 17
Check yourself . . . 20
Answers and solutions . . . 21

If you already know SQL you can use it on any DataFrame. The bridge is a **temporary view**: a name that lets SQL see your DataFrame as if it were a table.

7.1 createTempView()



Nothing is copied - a temp view is only a name for the DataFrame, and it lives only for this session.

`%sql cell`
the whole cell is SQL
Databricks shows the table

`spark.sql("...")`
SQL from Python
returns a DataFrame (df_sql)

PYSPARK

```
df.createTempView('my_view')
```

DECODE THE CODE

`.createTempView('name')`	Registers the DataFrame under a name so SQL can query it. Lives only for this session.

- `createTempView('my_view')` registers the DataFrame `df` under the name `my_view`. Nothing is copied - it is only a name for the DataFrame.
- The view lives only for the **current session**; when the notebook detaches or the cluster restarts it is gone.
- Running the same cell twice **fails** because the name already exists.

	Temporary view	Table (Phase 6)
Stores data?	No - just a name for a query / DataFrame	Yes - files on storage
Lives	This session only	Until dropped
Created by	`createTempView`	`saveAsTable`

RE-RUNNABLE VERSION

PYSPARK · EXTRA

```
df.createOrReplaceTempView('my_view')     # no error if my_view already exists
```

7.2 Running SQL: the %sql cell

SQL · A DATABRICKS %sql CELL

```
%sql

select * from my_view where Item_Fat_Content = 'Lf'
```

DECODE THE CODE

`%sql`	Databricks magic command: this one cell runs SQL instead of Python.
`select ... from`	Pick columns **from** a table or view.
`where`	Keep only rows that satisfy the condition.

The line `%sql` at the top switches this one cell from Python to SQL. The query returns every row of the view where the fat content is **Lf** and Databricks shows it as a table automatically.

> WHY 'LF' FINDS ROWS
> The original data says "Low Fat". The value **Lf** exists only because of the `regexp_replace` cell in Phase 2 that rewrote Low Fat to Lf before the view was created. If that cell was skipped, this query returns no rows.

7.3 spark.sql(): SQL that returns a DataFrame

PYSPARK

```
df_sql = spark.sql("select * from my_view where Item_Fat_Content = 'Lf'")
```

DECODE THE CODE

`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`spark.sql("...")`	Runs a SQL query from Python and returns a **DataFrame**.

PYSPARK

```
df_sql.display()
```

DECODE THE CODE

`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.

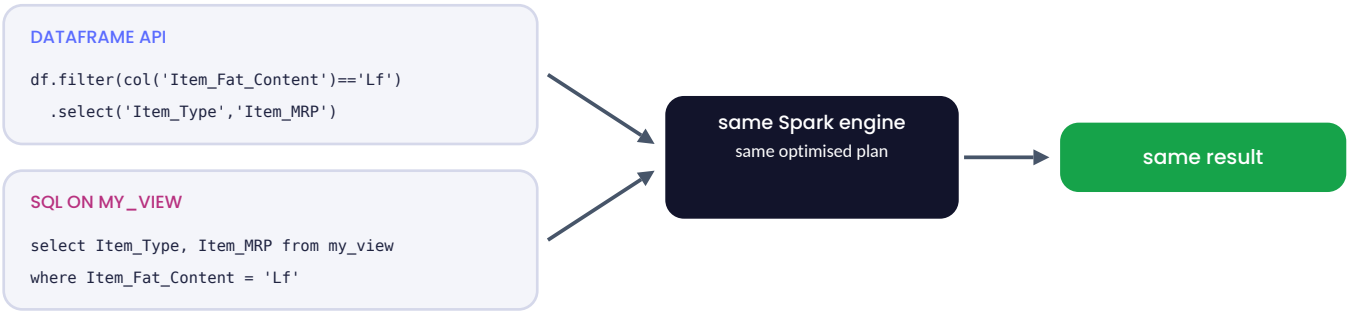
`spark.sql("...")` runs the same query from inside Python and gives you a normal **DataFrame**, so you can keep chaining DataFrame operations or write it out. This works anywhere (not only in Databricks), whereas the `%sql` magic is a notebook feature.

DataFrame API and SQL are two ways to say the same thing

Task	DataFrame API	SQL on my_view
Filter rows	`df.filter(col('Item_Fat_Content') == 'Lf')`	`WHERE Item_Fat_Content = 'Lf'`
Choose columns	`df.select('Item_Type', 'Item_MRP')`	`SELECT Item_Type, Item_MRP`
Group and total	`df.groupBy('Item_Type').agg(sum('Item_MRP'))`	`SELECT Item_Type, SUM(Item_MRP) ... GROUP BY Item_Type`
Sort	`df.sort(col('Item_MRP').desc())`	`ORDER BY Item_MRP DESC`
First n rows	`df.limit(10)`	`LIMIT 10`

WHICH ONE SHOULD I USE?

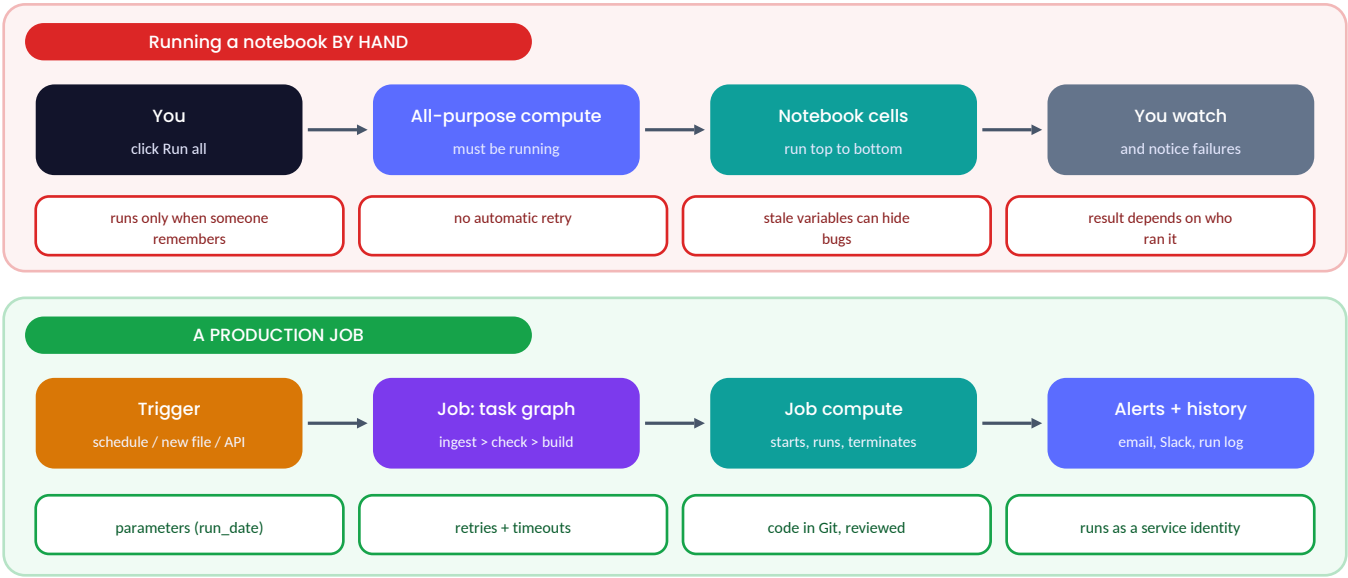
They run on the same engine and optimise to the same plan, so performance is the same. Use whichever reads best for the task - many teams write the heavy logic in SQL and glue it together in PySpark.



7.4 Running a notebook vs running in production

BEYOND THE NOTEBOOK

So far every cell was run **by you, by hand**, on a cluster you started. That is how you *develop*. A **production pipeline** runs by itself, on a schedule or a trigger, and must cope with failure without you watching.



	Running a notebook manually	Production pipeline (a Job)
Who starts it	You, with Run all.	A schedule, a file arrival, or another job.
Compute	All-purpose cluster you started.	Job compute or serverless, created for the run and then released.
Input	A file you uploaded.	Parameters (for example the load date) and live sources.
Order of cells	Whatever order you click - state can be stale.	Fixed: top to bottom, in tasks with dependencies.
If it fails	You see a red cell and fix it.	Retries, alerts, a run history, and a safe rerun.
Output	display() on screen.	Tables (Delta), logs and metrics - **no** display() needed.
Credentials	Your own.	A service principal / managed identity.

MAKE A NOTEBOOK PRODUCTION-READY

Remove `display()` and exploratory cells; keep logic in functions.

Replace hard-coded paths and dates with **parameters** (widgets, 7.5).

Make the write **idempotent** (7.9) so a rerun is safe.

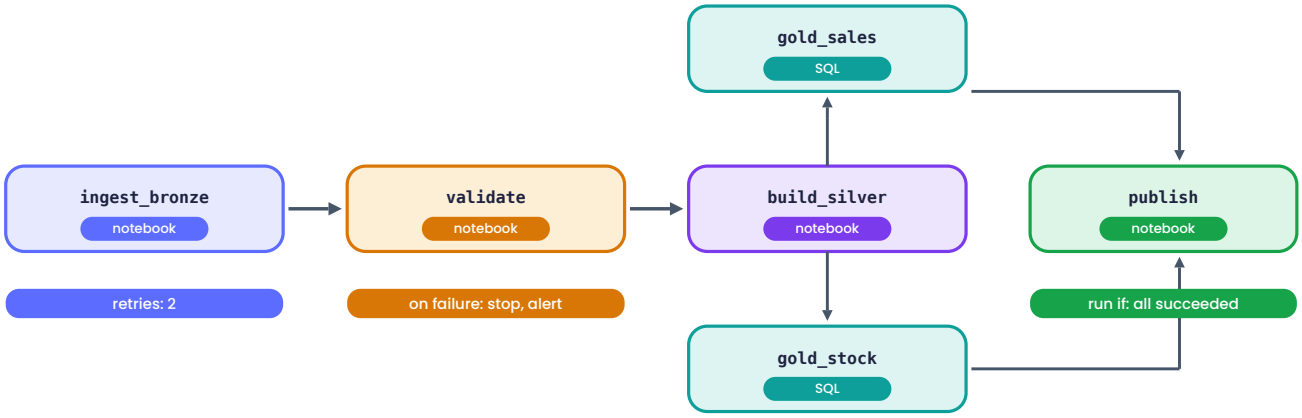
Fail loudly: let exceptions propagate so the job is marked **Failed** (7.10).

7.5 Databricks Jobs and orchestration

BEYOND THE NOTEBOOK

Orchestration means deciding *what runs, in what order, when, and what happens if a step fails*. On Databricks this is done by **Jobs** (now branded Lakeflow Jobs). A job is made of **tasks**; each task runs a notebook, a Python script, SQL or a pipeline, and tasks form a graph (DAG) of dependencies.

TASK GRAPH (a DAG): arrows mean 'depends on'



SETTINGS THAT MAKE IT A JOB, NOT JUST A NOTEBOOK

Trigger	Parameters	Compute	Notifications
cron schedule, file arrival, continuous, manual / API	run_date, env -> dbutils.widgets	job compute or serverless per task	email / webhook on start, success, failure

Job concept	Meaning
Task	One unit of work (a notebook, script, SQL statement, pipeline ...).
Dependency	Task B starts only after task A succeeds.
Trigger	Scheduled (cron), file arrival, continuous, or manual / API.
Parameters	Values passed into tasks, for example `load_date`.
Retries and timeouts	Re-run a failed task a few times; stop one that hangs.
Alerts	Email or webhook notification on start, success or failure.
Run history	Every run's status, duration, output and parameters.

Parameters and passing values between tasks

PYSPARK · WIDGETS AND TASK VALUES (DATABRICKS ONLY, ILLUSTRATIVE)

```
dbutils.widgets.text('load_date', '2026-10-15')           # a parameter with a default
load_date = dbutils.widgets.get('load_date')              # read it inside the notebook

# in the first task: publish a value for later tasks
dbutils.jobs.taskValues.set(key='row_count', value=df.count())

# in a later task: read it
n = dbutils.jobs.taskValues.get(taskKey='ingest', key='row_count', default=0)

dbutils.notebook.exit('OK')                               # end with a result string
```

DECODE THE CODE

`dbutils.widgets`	Notebook parameters: *text(name, default)* creates one, *get(name)* reads it. Jobs fill them in.
`dbutils.jobs.taskValues`	Pass small values from one job task to a later one.
`count(col)`	Number of rows / non-null values.
`dbutils.notebook.exit(text)`	Ends the notebook and returns a result string to the job.

JOB TASKS AND NOTEBOOKS

The job passes *parameters* to a notebook task, which shows up as a widget with the same name. Run manually, the widget simply uses its default.

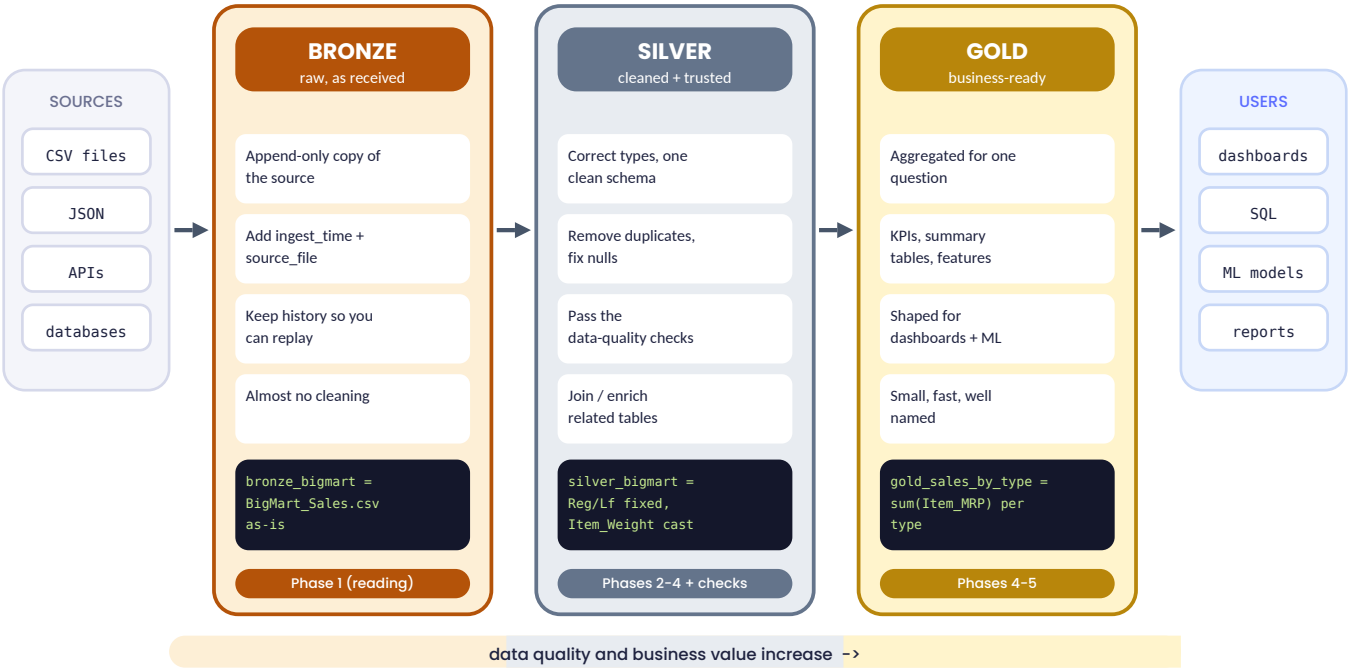
NAMES CHANGE

Databricks has renamed products over time: **Workflows** became **Lakeflow Jobs**, and **Delta Live Tables** became **Lakeflow Spark Declarative Pipelines**. The ideas (tasks, DAGs, triggers) stay the same. Outside Databricks the same role is played by tools such as Airflow or Azure Data Factory.

7.6 Medallion architecture: Bronze, Silver and Gold

BEYOND THE NOTEBOOK

The **medallion architecture** organises a lakehouse into three layers. Data gets **cleaner and more useful** at each step, and you can always trace a number back to the raw data.



Each layer is usually a Delta table. A problem found late can be fixed by re-running from Bronze, because Bronze keeps the original.

Layer	Purpose	Typical work	BigMart example
Bronze	Raw copy, as received. Append-only; replayable.	Read the file, add load metadata (file name, load time). **No** business fixes.	Raw BigMart CSV rows, all columns as received.
Silver	Clean, validated, de-duplicated, typed.	Apply the schema, convert markers to null (3.7), fix categories, drop duplicates, run DQ checks.	One typed row per Item + Outlet; `Lf` / `Regular` standardised.
Gold	Business-ready tables for reports and ML.	Joins and aggregations, one table per question.	Total sales by Outlet_Type and Item_Type.

PYSPARK · A THREE-LAYER PIPELINE FROM THE NOTEBOOK'S DATA (ILLUSTRATIVE)

```python
from pyspark.sql import functions as F

# BRONZE: land the raw file untouched, plus lineage columns
raw = (spark.read.format('csv').option('header', True)
            .load('/Volumes/main/sales/raw/BigMart_Sales.csv')
            .withColumn('_source_file', F.col('_metadata.file_path'))
            .withColumn('_loaded_at', F.current_timestamp()))
raw.write.format('delta').mode('append').saveAsTable('main.sales.bronze_bigmart')

# SILVER: clean, type, validate
bronze = spark.table('main.sales.bronze_bigmart')
silver = (bronze
          .withColumn('Item_Weight', F.expr('try_cast(Item_Weight as double)'))
          .withColumn('Item_Fat_Content',
                      F.regexp_replace('Item_Fat_Content', 'Low Fat|LF|low fat', 'Lf'))
          .dropDuplicates(['Item_Identifier', 'Outlet_Identifier']))
silver.write.format('delta').mode('overwrite').saveAsTable('main.sales.silver_bigmart')

# GOLD: an aggregate for the business
gold = (spark.table('main.sales.silver_bigmart')
             .groupBy('Outlet_Type', 'Item_Type')
             .agg(F.sum('Item_Outlet_Sales').alias('total_sales')))
gold.write.format('delta').mode('overwrite').saveAsTable('main.sales.gold_sales_by_type')
```

DECODE THE CODE

`functions as F`	Imports Spark's functions under the short name **F**, so you write *F.col*, *F.sum* ...
`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)
`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`_metadata.file_path`	A hidden Databricks column with the path of the file each row came from.
`current_timestamp()`	The current date and time - used to stamp rows with a load time.

ABOUT THE OUTPUTS IN THIS SECTION

Delta Lake examples and their outputs are **illustrative**: they follow the documented behaviour of Delta Lake / Databricks but were not executed while these notes were built. Run them in your own workspace to see the real values (version numbers, timestamps, file counts).

WHERE THE NOTEBOOK ALREADY IS

The notebook's steps map onto the layers: **read** = Bronze, **select / rename / cast / replace / dropDuplicates** = Silver, **groupBy / agg / joins** = Gold. It just does everything in one DataFrame variable and one notebook, without saving between layers.

WHY SEPARATE LAYERS AT ALL?

Reprocess Silver from Bronze when a rule changes - no need to go back to the source.

Isolate failures: a bug in Gold does not damage Bronze.

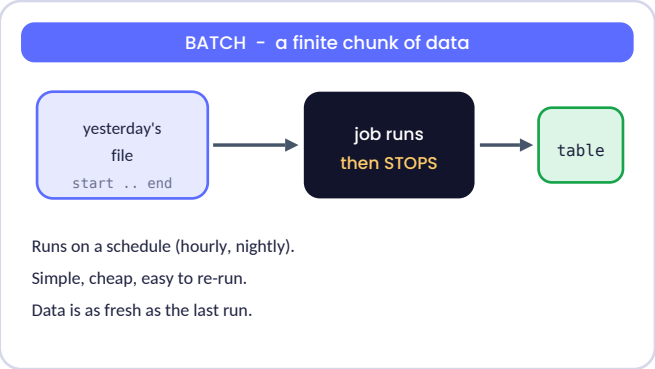
Different audiences: engineers use Bronze / Silver, analysts use Gold.

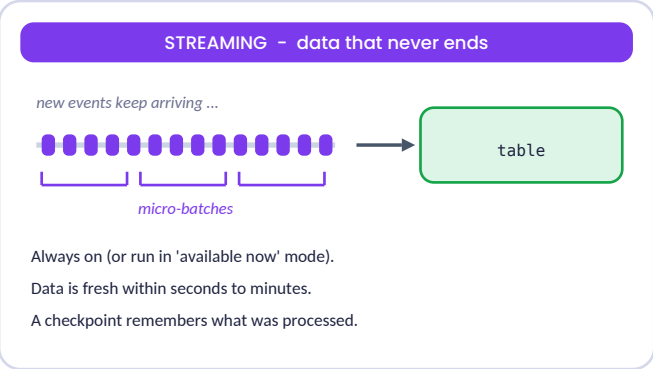
7.7 Batch vs streaming

BEYOND THE NOTEBOOK

	Batch	Streaming
Input	A bounded set (a file, yesterday's data).	An unbounded flow of new data.
Runs	Start, process everything, stop.	Keeps running (or runs in short bursts) and processes what arrives.
Latency	Minutes to hours.	Seconds to minutes.
Cost / complexity	Simpler and often cheaper.	More moving parts: checkpoints, state, monitoring.
Code	`spark.read` / `df.write`	`spark.readStream` / `df.writeStream`

Spark's **Structured Streaming** uses the **same DataFrame API**, treating new files or rows as an ever-growing table. The extra pieces are a **checkpoint** (so a restart continues where it stopped) and a **trigger** (when to process).






```
spark.readStream.format('cloudFiles')...        # Auto Loader: new files as a stream
.writeStream.option('checkpointLocation', cp)   # progress is saved here
.trigger(availableNow=True).toTable('bronze_t') # process what is new, then stop
```

PYSPARK · INCREMENTAL FILE INGESTION WITH AUTO LOADER (DATABRICKS, ILLUSTRATIVE)

```
stream = (spark.readStream.format('cloudFiles')            # Auto Loader: picks up NEW files only
               .option('cloudFiles.format', 'csv')
               .option('header', True)
               .option('cloudFiles.schemaLocation', '/Volumes/main/sales/chk/schema')
               .load('/Volumes/main/sales/raw/'))

(stream.writeStream
       .option('checkpointLocation', '/Volumes/main/sales/chk/bronze')
       .trigger(availableNow=True)                         # process what is there, then stop
       .toTable('main.sales.bronze_bigmart'))
```

DECODE THE CODE

`spark.readStream`	Reads data as a **stream**: new files / rows keep arriving.
`cloudFiles (Auto Loader)`	Databricks Auto Loader: picks up only **new** files in a folder.
`.writeStream`	Writes a stream out continuously (or in bursts).
`checkpointLocation`	Folder where the stream remembers what it has processed, so a restart continues.
`trigger(availableNow=True)`	Process everything new right now, then stop - streaming on a schedule.
`.toTable('name')`	Write the stream into a table.

AVAILABLENOW: STREAMING ON A SCHEDULE

`trigger(availableNow=True)` processes everything new and then stops, so a scheduled job gets streaming's **incremental, checkpointed** behaviour with batch-style cost. On serverless compute it is the only streaming trigger supported.

STREAMING IS NOT AUTOMATICALLY BETTER

Pick it when freshness really matters. Many pipelines are happier as a daily batch.

7.8 Full vs incremental loads

FULL load - copy everything

id	amount
1	10
2	20
3	30
4	40

ALL rows →

id	amount
1	10
2	20
3	30
4	40

Replace the target every run.

Simple and always correct.

Cost grows with the whole table.

INCREMENTAL load - only what changed

id	amount	updated
1	10	Oct 12
2	20	Oct 13
3	30	Oct 14
4	40	Oct 15

watermark = Oct 13

Read only rows newer than the watermark,

MERGE them in, then move the watermark.

Fast, but needs a reliable change signal.

HOW AN INCREMENTAL LOAD KNOWS WHAT IS NEW

- **Timestamp / ID watermark** — WHERE updated > last_loaded
- **New files only** — Auto Loader remembers files
- **Change data capture** — source sends inserts, updates, deletes
- **Partition by date** — load run_date's partition

BEYOND THE NOTEBOOK

	Full load	Incremental load
What it reads	The entire source every time.	Only new or changed rows since the last run.
Cost	Grows with total size.	Grows with change volume.
Simplicity	Very simple; no state to track.	Needs a **watermark** (last timestamp / id / file) and a merge.
Handles deletes?	Yes - the target equals the source.	Only if the source flags them.
Typical use	Small tables, reference data, first load.	Large, growing tables.

PYSPARK · A SIMPLE WATERMARK (ILLUSTRATIVE)

```
last = spark.sql('SELECT MAX(updated_at) FROM main.sales.silver_bigmart').first()[0]
last = last or '1900-01-01'                          # first run: no watermark yet

bronze = spark.table('main.sales.bronze_bigmart')
new_rows = bronze.filter(col('updated_at') > last)   # the increment
# ... clean new_rows, then MERGE into silver (6.9)
```

DECODE THE CODE

`spark.table('name')`	Reads a registered table as a DataFrame.
`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.

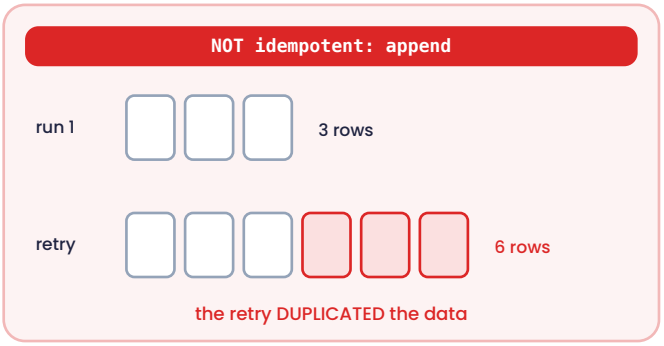
WHERE THE WATERMARK LIVES

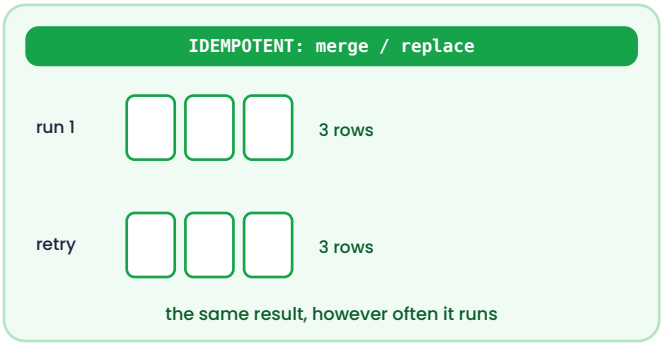
Read it from the target table (as above) or store it in a small *control table*. If it is kept only in a notebook variable, it is lost on the next run.

7.9 Idempotency and safe reruns

BEYOND THE NOTEBOOK

A step is **idempotent** if running it twice gives the same result as running it once. Pipelines fail and get rerun; without idempotency every rerun can duplicate or corrupt data.





FOUR WAYS TO MAKE A JOB SAFE TO RE-RUN

1 Overwrite a partition

```
replaceWhere run_date
= ...
```

each run owns its slice

2 MERGE on a key

```
update matches, insert
the rest
```

no duplicates by design

3 Deduplicate first

```
row_number() over key,
keep 1
```

clean before inserting

4 Write, then swap

```
output to a run_id
path
```

publish only on success

Pattern	Idempotent?	Why
`append` the same batch twice	**No**	Duplicate rows. This is the notebook's own gotcha from Phase 6.
`overwrite` the whole target	Yes	The end state is the same however often it runs.
`overwrite` one partition (`replaceWhere`)	Yes	Only that day's slice is replaced.
`MERGE` on a key	Yes	The second run matches every key and changes nothing.
Streaming with a checkpoint	Yes	The checkpoint remembers which files / offsets are done.
`createTempView`	No - it errors on rerun	Use `createOrReplaceTempView`.

A SAFE-RERUN CHECKLIST

Does the step **replace** or **merge** instead of blindly appending?

Is the batch identified by a **run id / load date** so you can delete or replace exactly it?

Can I rerun just the failed layer, starting from the previous layer's saved output?

7.10 Lineage, error handling and failed runs

BEYOND THE NOTEBOOK

Lineage answers *where did this number come from?* It records which sources, steps and tables produced each table. In Unity Catalog it is captured automatically for queries on registered tables (visible in Catalog Explorer). You support it by **adding metadata columns** and by using tables instead of loose paths.



TABLE LINEAGE: which objects feed which

COLUMN LINEAGE: WHERE ONE NUMBER COMES FROM



Where did this number come from?
trace Total_MRP back to the file

What breaks if I change a column?
see every downstream table

Who uses this table?
find the dashboards and jobs

Unity Catalog can capture table and column lineage automatically for supported queries. You add row-level lineage yourself:
`source_file, ingest_time and run_id columns on every Bronze row.`

PYSPARK · LINEAGE COLUMNS ON EVERY ROW

```
df = (df.withColumn('_source_file', F.col('_metadata.file_path'))   # which file?
        .withColumn('_loaded_at',   F.current_timestamp())          # when loaded?
        .withColumn('_run_id',      F.lit(run_id)))                 # which run?
```

DECODE THE CODE

`lit(value)`	Turns a plain Python value into a constant column (otherwise Spark looks for a column with that name).

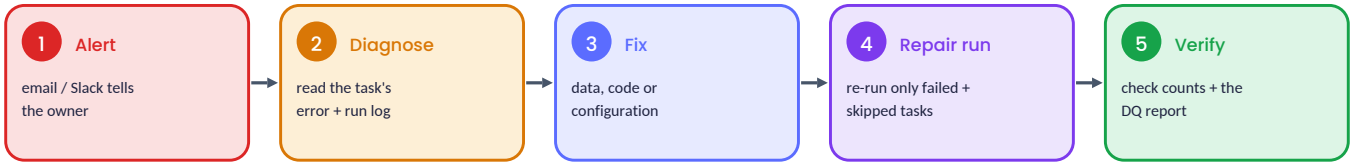
Error handling

The rule of thumb: **catch to record, then raise.** Catching an error and carrying on makes the job look green while the data is wrong.



A failed task stops everything that depends on it. Nothing wrong is published.

WHAT HAPPENS NEXT




```
try:
    run_pipeline(run_date)
except Exception as e:
    log_run(run_date, 'FAILED', str(e))   # keep a record
    raise                                 # let the JOB fail, loudly
```

PYSPARK · RECORD THE FAILURE, THEN RAISE (ILLUSTRATIVE)

```
from datetime import datetime

def log_run(step, status, message=''):
    schema = 'run_id STRING, step STRING, status STRING, message STRING, logged_at TIMESTAMP'
    spark.createDataFrame([(run_id, step, status, message, datetime.now())], schema)\
         .write.mode('append').saveAsTable('main.sales.pipeline_run_log')

try:
    fail_on(report, ['emp_id is unique'])          # from 4.12
    log_run('silver', 'SUCCESS')
except Exception as e:
    log_run('silver', 'FAILED', str(e)[:500])
    raise                                           # re-raise: the job must be marked Failed
```

DECODE THE CODE

`def name(args):`	Defines an ordinary Python function.
`STRING / DOUBLE / INT`	SQL-style type names: text, decimal number, whole number.
`spark.createDataFrame(rows, schema)`	Builds a DataFrame from Python data.
`spark.createDataFrame(data, schema)`	Builds a small DataFrame from a Python list plus a schema string.
`[('1','kad'), ...]`	A Python list of tuples. Each tuple is one row of the DataFrame.
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`df.write`	Starts a **DataFrameWriter** - the mirror image of spark.read.
`.mode('append')`	What to do if the destination already exists: error, append, overwrite or ignore.
`.saveAsTable('name')`	Saves the data as a **named table** in Spark's catalog.
`try: ... except`	Runs code and catches an error so you can record it. Re-**raise** afterwards so the failure is not hidden.

When a run fails ...	Do this
Read the error	The job run page shows which task failed and its output; the notebook cell shows the stack trace.
Decide: transient or real?	Network or cluster blips: let **retries** handle it. Data or logic errors need a fix.
Check the target	Delta's atomic commits mean the failed write left **no half result**; use `DESCRIBE HISTORY` to confirm.
Rerun	Repair-run only the failed task, or the whole job - safe only if the steps are idempotent (7.9).
Prevent	Add a DQ check or alert for the cause you found.

SILENT FAILURES ARE THE WORST FAILURES

A job that succeeds with 0 rows, or an inner join that quietly drops data, never raises an error. Check **row counts** and run the DQ checks from 4.12 before declaring success.

7.11 What the course notebook does and what is still ahead

BEYOND THE NOTEBOOK

The course notebook is a learning notebook: it implements the **PySpark skills** but not the full engineering around them. The table separates what the notebook **really contains** from what is **taught only in these notes** and what would be a **future improvement**. The last column has an interactive checkbox: tick the concepts that your own project already implements.

Concept	Status	Where	In my project
Read CSV with an explicit schema (DDL / StructType)	IN THE NOTEBOOK	Phase 1	☐
select, filter, withColumn, cast, sort, dropDuplicates	IN THE NOTEBOOK	Phase 2	☐
Null handling: fillna, dropna	IN THE NOTEBOOK	Phase 3	☐
groupBy, agg, pivot, joins, window functions, UDF	IN THE NOTEBOOK	Phases 4 - 5	☐
Write modes: append, overwrite, error, ignore	IN THE NOTEBOOK	Phase 6	☐
Managed and external tables, saveAsTable	IN THE NOTEBOOK	Phase 6	☐
Temp views and Spark SQL	IN THE NOTEBOOK	Phase 7	☐
Run by hand, cell by cell, on a cluster	IN THE NOTEBOOK	whole notebook	☐
explain(), stages, shuffles, repartition / coalesce, cache, broadcast	TAUGHT IN THESE NOTES	0.10, 4.6 - 4.10	☐
Schema validation and drift checks	TAUGHT IN THESE NOTES	1.8	☐
Null markers such as \N	TAUGHT IN THESE NOTES	3.7	☐
Reusable data-quality checks, referential integrity	TAUGHT IN THESE NOTES	4.11, 4.12	☐
Delta tables, merge, time travel, constraints	TAUGHT IN THESE NOTES	6.6 - 6.11	☐
Unity Catalog, volumes, managed vs external	TAUGHT IN THESE NOTES	6.12	☐
Bronze / Silver / Gold layers saved as tables	FUTURE STEP	7.6	☐
Incremental loads with a watermark	FUTURE STEP	7.8	☐
Idempotent writes (merge) and safe reruns	FUTURE STEP	7.9	☐
Streaming / Auto Loader ingestion	FUTURE STEP	7.7	☐
Lineage columns, run log, alerts	FUTURE STEP	7.10	☐
Scheduled Jobs with retries and parameters	FUTURE STEP	7.4, 7.5	☐
Automated tests of transformations	FUTURE STEP	-	☐
Environments (dev / prod) and CI/CD	FUTURE STEP	-	☐

HOW TO READ THE STATUS

In the notebook = demonstrated by the course notebook. **Taught in these notes** = beyond the notebook, but explained here. **Future step** = a sensible improvement when you turn practice into a production pipeline. The statuses describe the **course notebook**; your own project may already do more - that is what the checkboxes are for.

INTERVIEW CORNER

Q What is the medallion architecture?

A A layered design: Bronze (raw, append-only), Silver (cleaned, validated, typed), Gold (aggregated, business-ready). Quality and business value rise at each layer, and each can be rebuilt from the one before.

Q Why not just clean the data on the way in?

A Keeping raw data in Bronze lets you reprocess when rules change, audit what arrived, and recover from bugs without re-fetching the source.

Q Batch vs streaming - how do you choose?

A By the freshness the business needs. Batch is simpler and cheaper; streaming gives low latency. availableNow streaming gives incremental processing on a schedule.

Q Full vs incremental load?

A Full reloads everything and is simple but grows with data size; incremental reads only changes using a watermark and merges them, but needs state and delete handling.

Q What is idempotency and how do you achieve it?

A Running a step twice gives the same result as once. Use overwrite, replaceWhere or MERGE on a key instead of blind append, identify each run, and use streaming checkpoints.

Q A nightly job failed at 3 a.m. What do you do?

A Read the failing task's error, decide whether it is transient or a data / logic problem, check that Delta left no partial write, fix the cause, repair-run the failed task, and add a check or alert so it is caught next time.

Q How is a Databricks Job different from running a notebook?

A A job runs unattended on job or serverless compute, with a trigger, parameters, task dependencies, retries, alerts and a run history; a notebook run is manual and interactive.

Q What is data lineage and why does it matter?

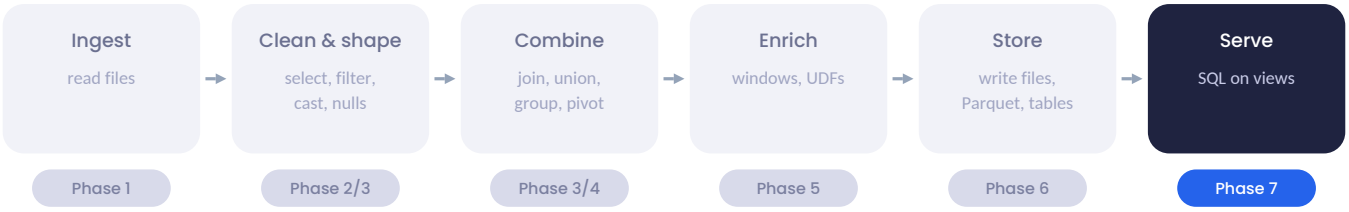
A The recorded path of data from source to report. It supports debugging, impact analysis and audit. Unity Catalog captures it for registered tables; metadata columns add row-level traceability.

PHASE 7 CHECKPOINT

- I can register a DataFrame with createTempView and know it is session-scoped.
- I can query it from a `%sql` cell and from `spark.sql(...)`.
- I know spark.sql returns a DataFrame I can keep transforming.
- I can translate between the DataFrame API and SQL.
- I can explain how a production job differs from a manual notebook run, and what a Job is made of.
- I can describe Bronze, Silver and Gold and place each notebook step in a layer.
- I can choose between batch and streaming, and between full and incremental loads.
- I can make a write idempotent, add lineage columns, and handle errors by logging then raising.
- I can separate what the course notebook implements from what is a future improvement.

From notebook to pipeline

Data engineers use these same skills inside **pipelines**. Here is where this phase sits, and how it is used in practice.



DATA ENGINEERING LENS

Temp views let SQL users query a pipeline's result without writing Python.

A temp view lives only for the session; a table saved in Phase 6 persists after the session ends.

spark.sql() returns a DataFrame, so SQL steps and DataFrame steps can be mixed in the same pipeline.

Beyond the notebook: production means **Jobs, parameters, retries, alerts** and idempotent writes - not Run all by hand.

Organise data as **Bronze (raw) - Silver (clean) - Gold (business)** so any layer can be rebuilt from the one before.

Glossary

Concepts from this phase, then every **code term** the decoder explained, in the order you met them.

Concept	In plain words
Temporary view	A session-only name that lets SQL query a DataFrame.
%sql	Databricks magic command that turns a cell into SQL.
spark.sql()	Runs a SQL query from Python and returns a DataFrame.
Medallion architecture	Bronze (raw), Silver (clean), Gold (business-ready) layers.
Batch / streaming	Bounded data processed in one go / unbounded data processed as it arrives.
Full / incremental load	Reload everything / load only new or changed rows.
Idempotent	Running twice gives the same result as running once.
Lineage	The recorded path of data from source to report.
Job / task	A scheduled unit of orchestration / one step (notebook, script, SQL) in it.
Watermark	The last timestamp or id processed, used to find new rows.

Code terms decoded in this phase

Code term	Meaning
`.createTempView('name')`	Registers the DataFrame under a name so SQL can query it. Lives only for this session.
`%sql`	Databricks magic command: this one cell runs SQL instead of Python.
`select ... from`	Pick columns **from** a table or view.
`where`	Keep only rows that satisfy the condition.
`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`spark.sql("...")`	Runs a SQL query from Python and returns a **DataFrame**.
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.
`dbutils.widgets`	Notebook parameters: *text(name, default)* creates one, *get(name)* reads it. Jobs fill them in.
`dbutils.jobs.taskValues`	Pass small values from one job task to a later one.
`count(col)`	Number of rows / non-null values.
`dbutils.notebook.exit(text)`	Ends the notebook and returns a result string to the job.
`functions as F`	Imports Spark's functions under the short name **F**, so you write *F.col*, *F.sum* ...
`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)
`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`_metadata.file_path`	A hidden Databricks column with the path of the file each row came from.
`current_timestamp()`	The current date and time - used to stamp rows with a load time.
`spark.readStream`	Reads data as a **stream**: new files / rows keep arriving.
`cloudFiles (Auto Loader)`	Databricks Auto Loader: picks up only **new** files in a folder.
`.writeStream`	Writes a stream out continuously (or in bursts).
`checkpointLocation`	Folder where the stream remembers what it has processed, so a restart continues.
`trigger(availableNow=True)`	Process everything new right now, then stop - streaming on a schedule.
`.toTable('name')`	Write the stream into a table.
`spark.table('name')`	Reads a registered table as a DataFrame.
`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.

Code term	Meaning
`lit(value)`	Turns a plain Python value into a constant column (otherwise Spark looks for a column with that name).
`def name(args):`	Defines an ordinary Python function.
`STRING / DOUBLE / INT`	SQL-style type names: text, decimal number, whole number.
`spark.createDataFrame(rows, schema)`	Builds a DataFrame from Python data.
`spark.createDataFrame(data, schema)`	Builds a small DataFrame from a Python list plus a schema string.
`[('1','kad'), ...]`	A Python list of tuples. Each tuple is one row of the DataFrame.
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`df.write`	Starts a **DataFrameWriter** - the mirror image of spark.read.
`.mode('append')`	What to do if the destination already exists: error, append, overwrite or ignore.
`.saveAsTable('name')`	Saves the data as a **named table** in Spark's catalog.
`try: ... except`	Runs code and catches an error so you can record it. Re-**raise** afterwards so the failure is not hidden.

Check yourself

Try each question first, then click **Show answer**. Every answer has a **back** link that returns you to the question.

Questions

Q1 What does `df.createTempView('my_view')` do and how long does it last? Show answer

Q2 What is the difference between a `%sql` cell and `spark.sql(...)`? Show answer

Q3 Why does the notebook's query `where Item_Fat_Content = 'Lf'` find rows? Show answer

Q4 What happens if you run `createTempView('my_view')` twice, and how do you avoid it? Show answer

Q5 Write `df.groupBy('Item_Type').agg(avg('Item_MRP'))` as SQL. Show answer

Q6 Name three differences between running a notebook by hand and a production job. Show answer

Q7 What does each medallion layer hold? Show answer

Q8 Why is append not idempotent, and what is? Show answer

Q9 A nightly job failed. What are your first steps? Show answer

Hands-on tasks

Task 1 In a %sql cell, show the five highest Item_MRP rows among the 'Lf' items. Show solution

Task 2 With spark.sql, count the rows for each Outlet_Size and display the result. Show solution

Task 3 Add lineage columns to a DataFrame before writing it to a Bronze table. Show solution

Answers and solutions

A1 It registers the DataFrame under the name my_view so SQL can query it. It lasts only for the current session and copies no data. back to question

A2 %sql turns the whole cell into SQL (a Databricks feature). spark.sql runs SQL from Python and returns a **DataFrame**. back to question

A3 Because the earlier regexp_replace cell rewrote 'Low Fat' to 'Lf' before the view was created. back to question

A4 The second run fails because the name exists. Use `createOrReplaceTempView`. back to question

A5 `select Item_Type, avg(Item_MRP) from my_view group by Item_Type` back to question

A6 Triggered automatically, runs on job / serverless compute with parameters, and has retries, alerts and run history. back to question

A7 Bronze: raw data as received. Silver: cleaned, typed, de-duplicated. Gold: aggregated, business-ready tables. back to question

A8 A rerun adds the same rows again. Overwrite, replaceWhere and MERGE on a key give the same result however often they run. back to question

A9 Read the failed task's error, decide transient vs real, confirm the target has no partial write (Delta history), fix, repair-run the failed task, and add a check or alert. back to question

Task solutions

Solution 1 back to task

SOLUTION

```
%sql

select * from my_view
where Item_Fat_Content = 'Lf'
order by Item_MRP desc
limit 5
```

Solution 2 back to task

SOLUTION

```
df_cnt = spark.sql("select Outlet_Size, count(*) as n from my_view group by Outlet_Size")

df_cnt.display()
```

Solution 3 back to task

SOLUTION

```
df = (df.withColumn('_source_file', F.col('_metadata.file_path'))
        .withColumn('_loaded_at', F.current_timestamp()))
df.write.format('delta').mode('append').saveAsTable('main.sales.bronze_bigmart')
```

PREVIOUS ← Phase 6 · Writing Data & Delta Lake

NEXT Appendices →

Tip: keep all nine PDFs in the same folder so the previous / next links open the right file.